Ridge Regression, Gradient Descent, And Perceptron

CSAI 253 - Spring 2024

Prepared By

Dr. Khaled Mostafa ElSayed

kmelsayed@zewailcity.edu.eg

Instructors:

Dr. Khaled Mostafa ElSayed

kmelsayed@zewailcity.edu.eg

Dr. Mohamed Maher Ata

momaher@zewailcity.edu.eg

ZEWAIL CITY

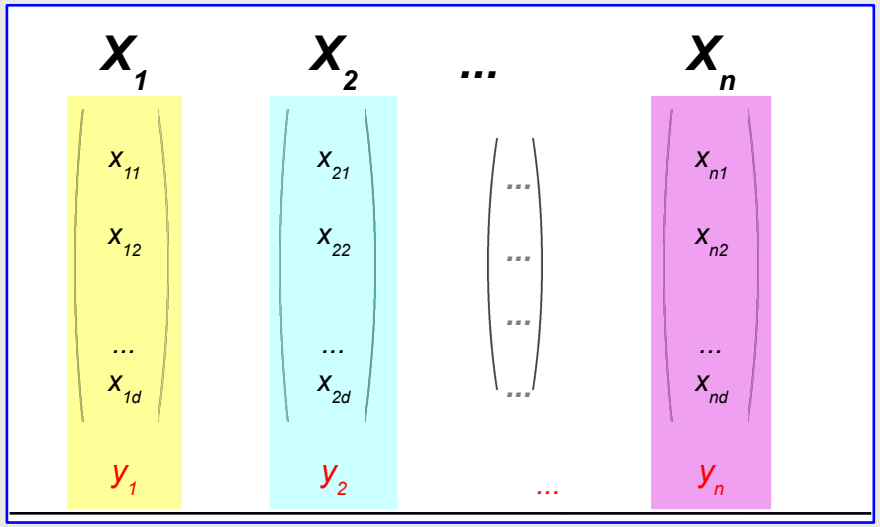
جامعة العلوم والتكنولوجيا

University of Science and Technology

Revision of Standard Linear Regression Model

Matrix Notation for Linear Regression and Classification (N- Samples)

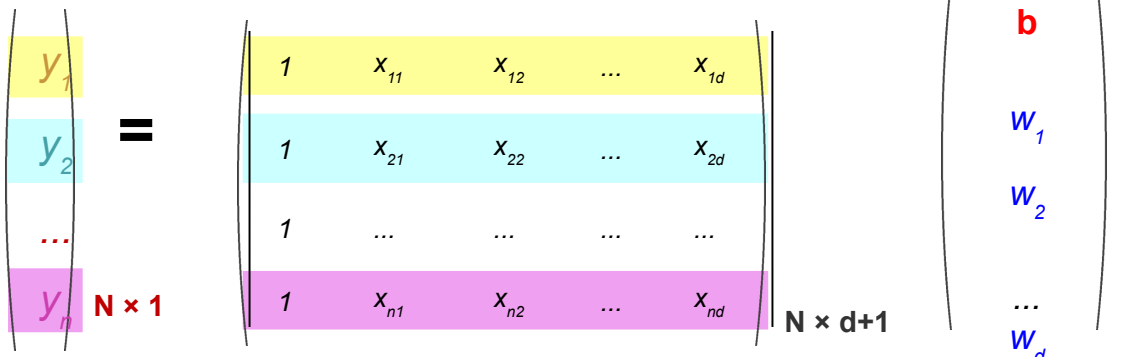
N Training Samples



Remember

$$\underline{y} = \underline{W}^T\underline{X} = \underline{X}^T\underline{W}$$

In General: **N >>> d**



$$\underline{Y} = \underline{X}\,\underline{W}$$

Where: $\underline{X}_{N\times d+1}$ **is a matrix**

$\underline{Y}_{N\times 1}$ **is labels vector**

Concept of LOSS Function

$$Let\ \underline{\mathbf{Y}} = \begin{pmatrix} y_1 \\ y_2 \\ \dots \\ y_n \end{pmatrix}_{N \times 1}$$

Original Labels

from Training Samples

$$Let\ \underline{\hat{\mathbf{Y}}} = \begin{pmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \dots \\ \hat{y}_n \end{pmatrix}_{N \times 1}$$

Predicted Labels

from Model $\hat{Y}=X\underline{W}$

Remember

$$\|a\|_2 = a^T a$$

Objective is $\hat{Y}=Y$

If $\hat{Y} \neq Y \Rightarrow$ There is an **Error (Loss)**

$\underline{\hat{\mathbf{Y}}} = \boldsymbol{X}\ \underline{\boldsymbol{W}}$ ***Predicted***

Where : $\boldsymbol{X}_{N \times d+1}$ is a **matrix** $\Rightarrow$ **Given**

$\underline{\boldsymbol{Y}}_{N \times 1}$ is labels **vector** $\Rightarrow$ **Given**

$\underline{\boldsymbol{W}}$ is Model Parameter $\Rightarrow$ **Required**

Define **Loss** function as **squared distance** between $\hat{Y}$ and Y

$$Loss = \|Y - \hat{\mathbf{Y}}\|_2 = (Y - \hat{\mathbf{Y}})^T (Y - \hat{\mathbf{Y}})$$

Minimization of Loss function to obtain BEST model parameters

$$Loss = \frac{1}{2}(Y - \hat{Y})^T(Y - \hat{Y}) = \frac{1}{2}(Y - XW)^T(Y - XW)$$

$$Loss = \frac{1}{2}[Y^TY + (XW)^T(XW) - Y^T(XW) - (XW)^TY]$$

$$Loss = \frac{1}{2}[Y^TY + (XW)^T(XW) - 2(XW)^TY]$$

Remember

X is a matrix, W is a vector,

XW is a vector

$$Y^T(XW) = (XW)^TY$$

$\frac{\partial Loss}{\partial W} = 0$ Get Best **W** that minimizes the loss

$$\frac{\partial Loss}{\partial W} = 0 = \frac{1}{2}[(XW)^T(X) + (X)^T(XW) - 2(X)^TY]$$

Remember

$$X^T(XW) = (XW)^TX$$

$$\frac{\partial Loss}{\partial W} = 0 = \frac{1}{2}[2X^TXW - 2X^TY] \rightarrow (X^TX)\underline{W} = X^TY \rightarrow$$

$$W = (X^TX)^{-1}X^TY$$

Problems of Linear Regression

(With Linearly Separable Data)

Problems of Linear Regression

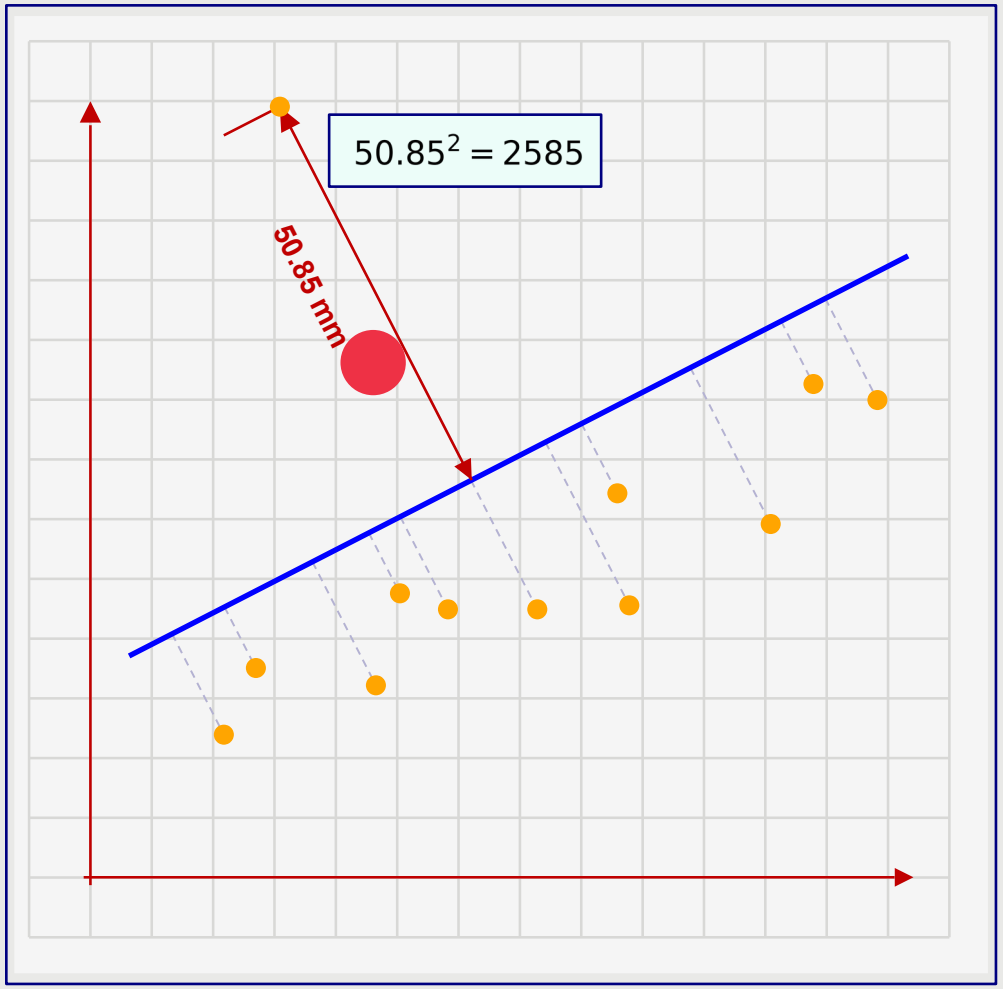
Assume the Dataset is linearly separable (suitable to apply Linear regression model)

Linear regression model has FOUR problems:

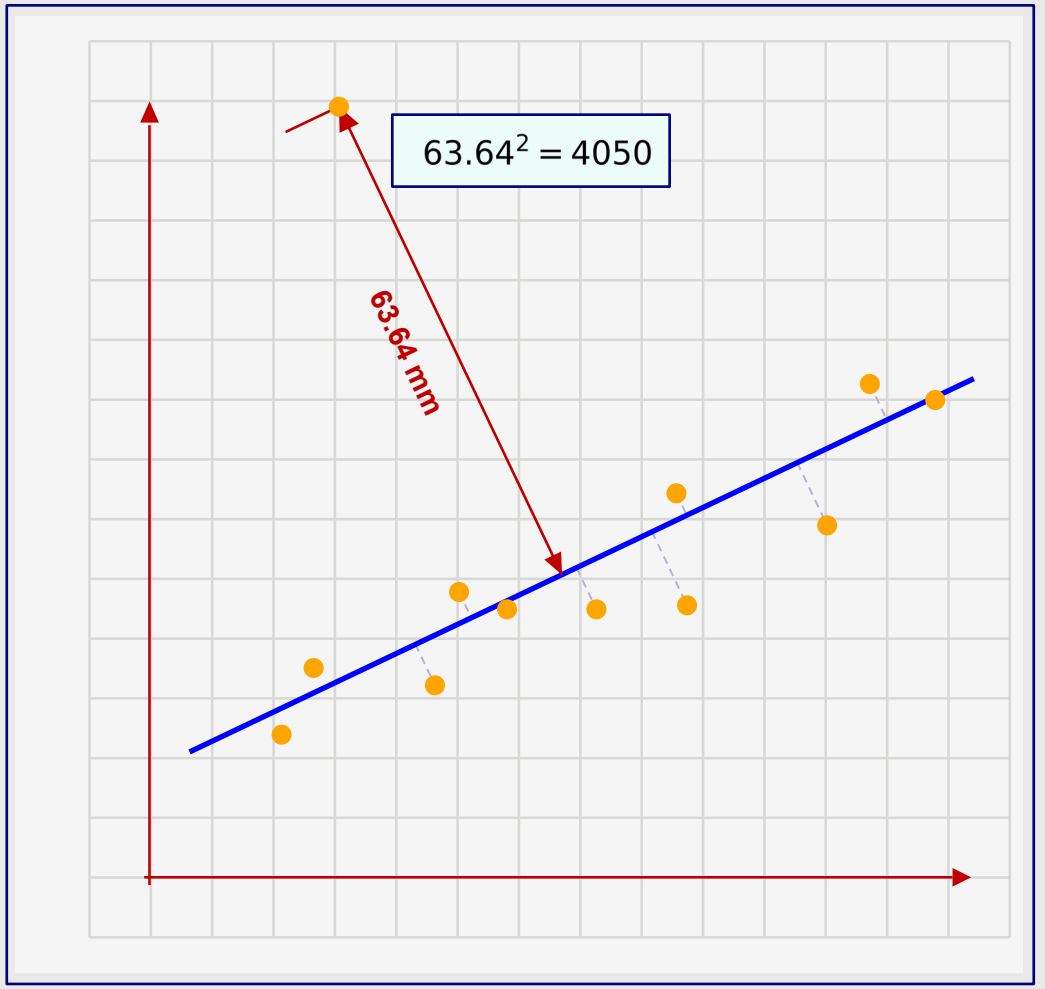
- **Overfitting Problem**
- **Matrix Inversion Problem [1]** ***(Singular Matrix)***
- **Matrix Inversion Problem [2]** ***(Large Matrix dimension)***
- **Different Hyperplanes can solve the problems** ***(Selection of Best Hyperplane)***

[1] Ridge Regularizer to Minimize Overfitting

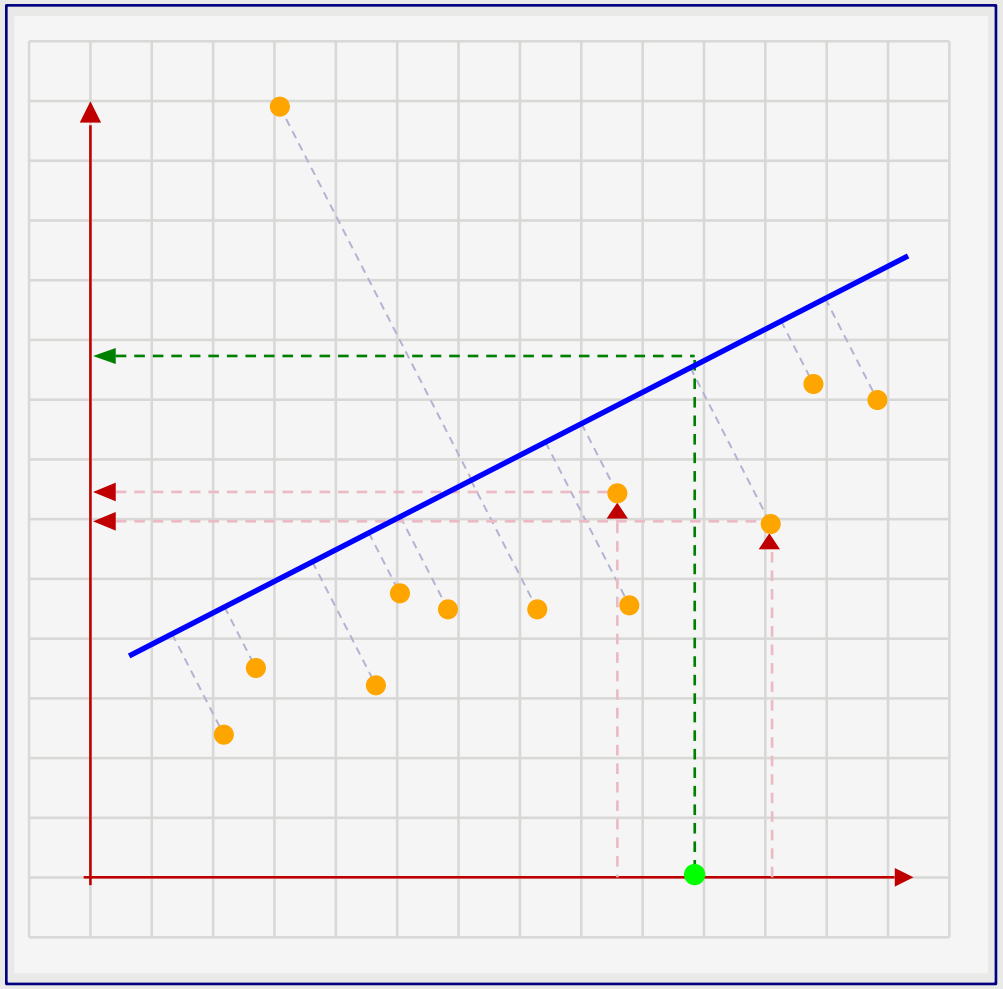
(Also Known as L2 regularizer)



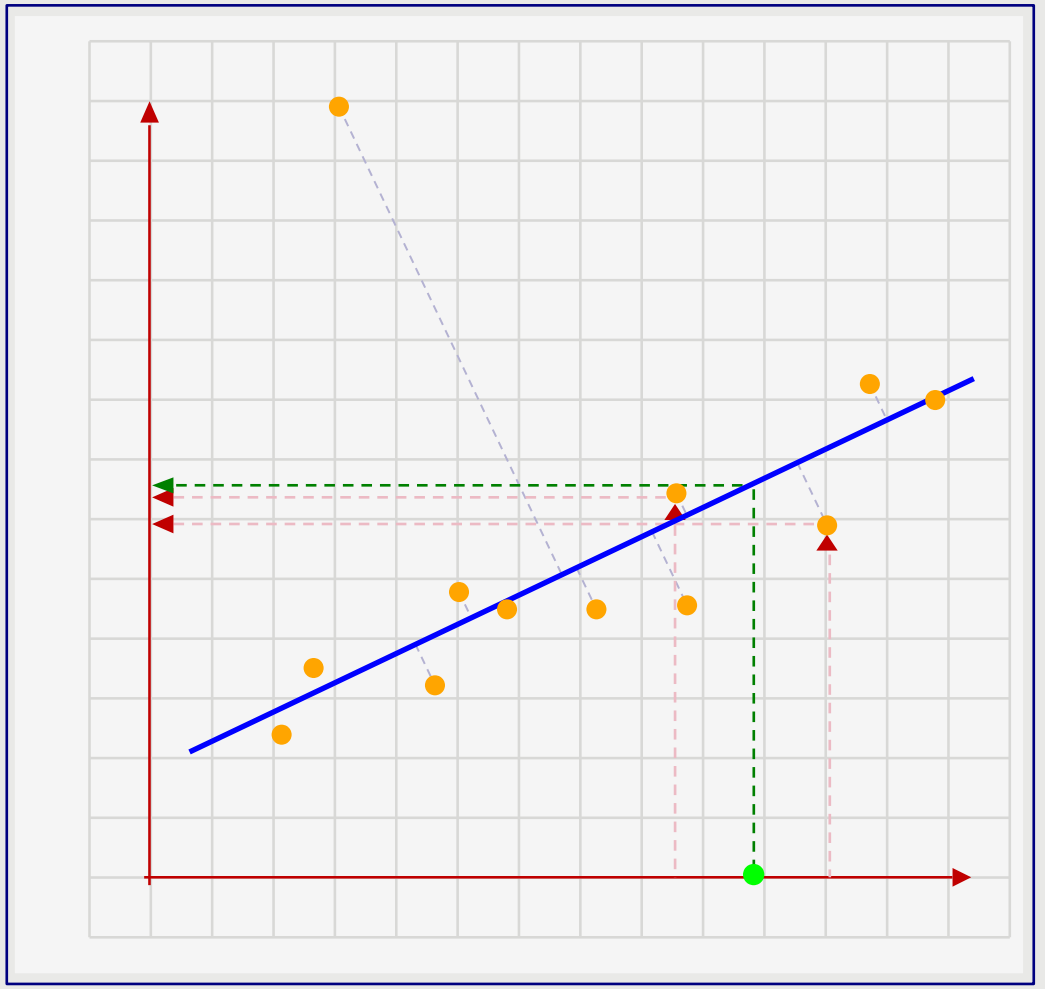
Overfit (try to minimize the residuals ONLY)
Fails to detect the Pattern



Generalized (Residuals are not minimized)
Successes to detect the Pattern



Overfit (try to minimize the residuals ONLY)
Fails to detect the Pattern



Generalized (Residuals are not minimized)
Successes to detect the Pattern

Example of Overfitting due to outlier in data

(and how to minimize overfitting)

Effect of Overfitting on model parameter "W"

Training Data

Perfect Data
Y follows x1

	X1	X2	X3	X4	X5	X6	X7	X8
X1	1	2	3	4	8	9	10	11
X2	1	3	5	6	16	7	5	3
y	2	4	6	8	16	18	20	22

X5 is Outlier

	X1	X2	X3	X4	X5	X6	X7	X8
X1	1	2	3	4	8	9	10	30
X2	1	3	5	6	16	7	5	3
y	2	4	6	8	16	18	20	22

X5 is Outlier

	X1	X2	X3	X4	X5	X6	X7	X8
X1	1	2	3	4	8	9	10	60
X2	1	3	5	6	16	7	5	3
y	2	4	6	8	16	18	20	22

Test Data

	Test1	Test2	Test3
X1	30	40	50
X2	15	20	25
y	60	80	100

Again
Y follows x1

What will be the effect of X8 Outlier values on the trained model ???

Effect of Overfitting on model parameter "W"

Train

	X_1	X_2	X_3	X_4	X_5	X_6	X_7	X_8
X_1	1	2	3	4	8	9	10	11
X_2	1	3	5	6	16	7	5	3
y	2	4	6	8	16	18	20	22

$$W = \begin{pmatrix} 0 \\ 2 \\ 0 \end{pmatrix}$$

$$\|W\|^2 = 4$$

Test

$Test_1$	$Test_2$	$Test_3$
30	40	50
15	20	25
60	80	100

	X_1	X_2	X_3	X_4	X_5	X_6	X_7	X_8
X_1	1	2	3	4	8	9	10	30
X_2	1	3	5	6	16	7	5	3
y	2	4	6	8	16	18	20	22

$$W = \begin{pmatrix} 2.7 \\ 0.67 \\ 0.64 \end{pmatrix}$$

$$\|W\|^2 = 8.11$$

$Test_1$	$Test_2$	$Test_3$
30	40	50
15	20	25
32.4	42.3	52.3

	X_1	X_2	X_3	X_4	X_5	X_6	X_7	X_8
X_1	1	2	3	4	8	9	10	60
X_2	1	3	5	6	16	7	5	3
y	2	4	6	8	16	18	20	22

$$W = \begin{pmatrix} 4 \\ 0.29 \\ 0.78 \end{pmatrix}$$

$$\|W\|^2 = 16.7$$

$Test_1$	$Test_2$	$Test_3$
30	40	50
15	20	25
24.4	31.2	38

pending recreation

pending recreation

L_1 and L_2 Regularization

Regularizers are based on:

Penalize large values of W (model parameters)

- This have the effect of better generalization of the model
- Since there is a contradiction between model training (using training dataset) and model generalization, the generalization term is added to the objective loss function with a customized weigh factor (do regularization to what extent)
- Penalization may be applied on the Squared Norm of the Weight Vector
  This is called L_2 regularization (also know as Ridge Regularization)
- Penalization may also be applied on the sum of Absolute values of the Weight Vector
  This is called L_1 regularization
- Regularization is particularly important for Small number of samples (N) and Large vector Length (d)

Adding Ridge Regularizer to the model λ=3

Train

	X_1	X_2	X_3	X_4	X_5	X_6	X_7	X_8
X_1	1	2	3	4	8	9	10	11
X_2	1	3	5	6	16	7	5	3
y	2	4	6	8	16	18	20	22

$$W = \begin{pmatrix} 0.1 \\ 1.95 \\ 0.03 \end{pmatrix}$$

$$\|W\|^2 = 3.8$$

Test

	$Test_1$	$Test_2$	$Test_3$
X_1	30	40	50
X_2	15	20	25
y	59	78.7	98.4

	X_1	X_2	X_3	X_4	X_5	X_6	X_7	X_8
X_1	1	2	3	4	8	9	10	30
X_2	1	3	5	6	16	7	5	3
y	2	4	6	8	16	18	20	22

$$W = \begin{pmatrix} 1.2 \\ 0.72 \\ 0.77 \end{pmatrix}$$

$$\|W\|^2 = 2.45$$

	$Test_1$	$Test_2$	$Test_3$
X_1	30	40	50
X_2	15	20	25
y	34.1	45.1	56.1

	X_1	X_2	X_3	X_4	X_5	X_6	X_7	X_8
X_1	1	2	3	4	8	9	10	60
X_2	1	3	5	6	16	7	5	3
y	2	4	6	8	16	18	20	22

$$W = \begin{pmatrix} 1.77 \\ 0.32 \\ 0.99 \end{pmatrix}$$

$$\|W\|^2 = 4.2$$

	$Test_1$	$Test_2$	$Test_3$
X_1	30	40	50
X_2	15	20	25
y	26.2	34.4	42.5

Adding Ridge Regularizer to the model λ=10

Train

	X_1	X_2	X_3	X_4	X_5	X_6	X_7	X_8
X_1	1	2	3	4	8	9	10	11
X_2	1	3	5	6	16	7	5	3
y	2	4	6	8	16	18	20	22

$$\underline{W} = \begin{pmatrix} 0.13 \\ 1.87 \\ 0.09 \end{pmatrix}$$

$$\|W\|^2 = 3.5$$

Test

	$Test_1$	$Test_2$	$Test_3$
X_1	30	40	50
X_2	15	20	25
y	57.4	76.5	95.6

	X_1	X_2	X_3	X_4	X_5	X_6	X_7	X_8
X_1	1	2	3	4	8	9	10	30
X_2	1	3	5	6	16	7	5	3
y	2	4	30	8	16	18	20	22

$$\underline{W} = \begin{pmatrix} 0.54 \\ 0.73 \\ 0.81 \end{pmatrix}$$

$$\|W\|^2 = 1.48$$

	$Test_1$	$Test_2$	$Test_3$
X_1	30	40	50
X_2	15	20	25
y	34.6	46.4	57.4

	X_1	X_2	X_3	X_4	X_5	X_6	X_7	X_8
X_1	1	2	3	4	8	9	10	60
X_2	1	3	5	6	16	7	5	3
y	2	4	60	8	16	18	20	22

$$\underline{W} = \begin{pmatrix} 0.82 \\ 0.33 \\ 1.1 \end{pmatrix}$$

$$\|W\|^2 = 1.92$$

	$Test_1$	$Test_2$	$Test_3$
X_1	30	40	50
X_2	15	20	25
y	26.8	35.4	44.1

pending recreation

[2] Ridge Regularization to overcome

Inverse Problem of Singular Matrix

Inverse Problem of Singular Matrix

$X^T X$

14	10	15
10	40	46
20	80	92

$|X^T X| = 0$ ⇒ Singular Matrix ⇒ *No Inverse*

$(X^T X + \lambda I)$ $\lambda = 3$

17	10	15
10	43	46
20	80	95

$|X^T X| = 5685$ ⇒ Non-Singular Matrix ⇒ *Inverse Exists*

$(X^T X + \lambda I)^{-1}$ ⇒

0.07	0.04	-0.03
-0.01	0.23	-0.11
-0.01	-0.2	0.11

[3] Inverse of High Dimension (non-singular) Matrix

Inverse of High Dimension Matrix

Recall

$$\underline{W} = (X^T X + \lambda I)^{-1} X^T \underline{Y}$$

if Feature vectors are of dimension d $\Rightarrow$ ($\underline{X}^T X$) is of size d+1 x d+1

The standard matrix inversion algorithm has a time complexity of $O(n^3)$, which means that the computation time increases rapidly with the size of the matrix (increasing cubically with the number of components).

For Example: if Features vector is of size 1000 then Inverse calculation is or order 1000,000,000 which is impractical

Solution:

*Instead of using **Least Squares** with Matrix inversion,*
*use **Gradient Descent** iterative solution*

Gradient Descent (GD)

and

How to apply to minimize Loss function

pending recreation

Gradient Descent Solution

Gradient Descent (GD) Algorithm:

[1] Start with initial **Random Weights (W)**

[2] Update the weights by moving along the direction of **negative gradient** of Loss function (w.r.t. W) (**moving step is a hyper parameter, AKA Learning Rate**)

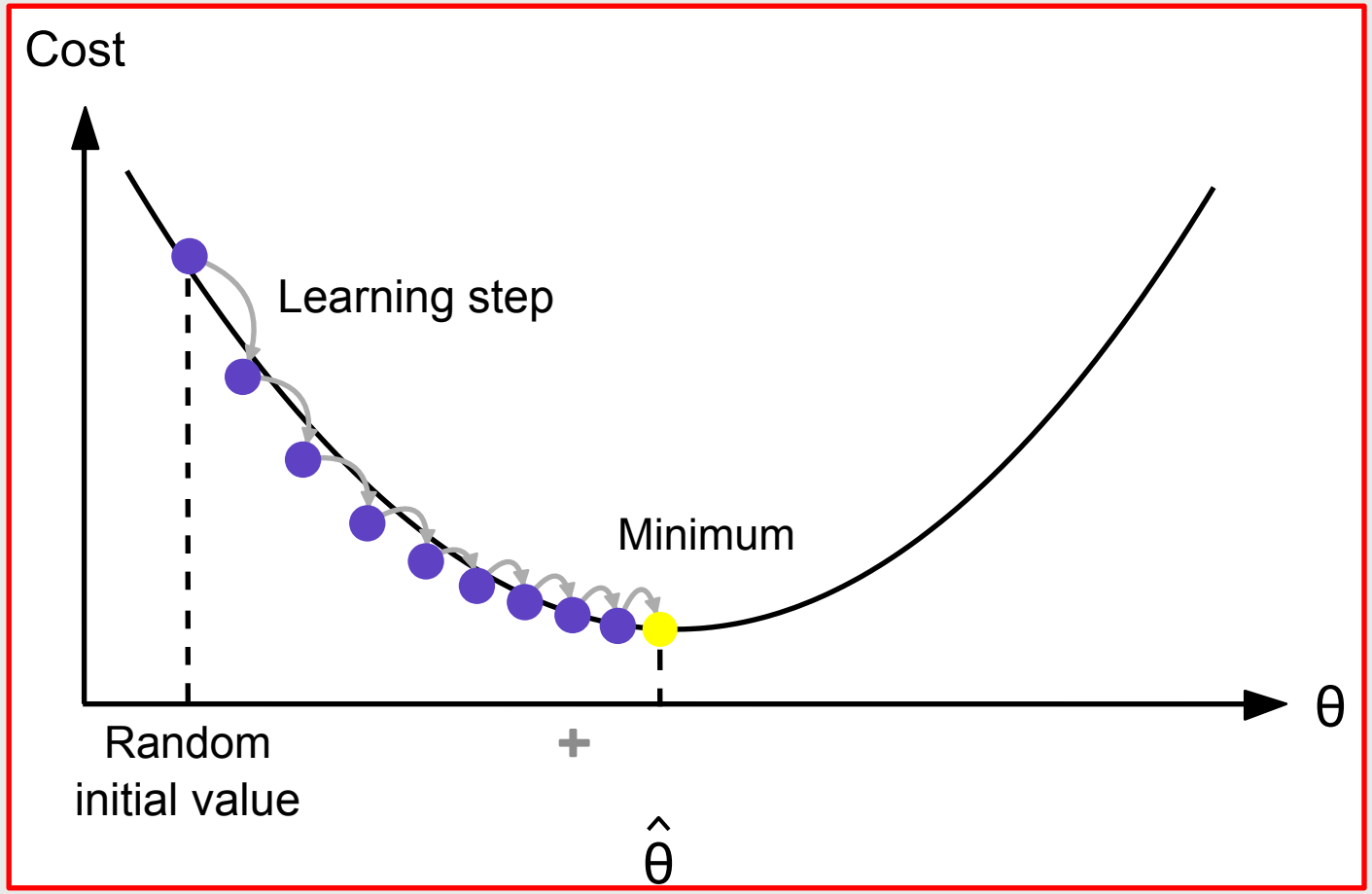
[3] Stop when the gradient is **almost zero** (this is at the minimum value of the function)

Notes:

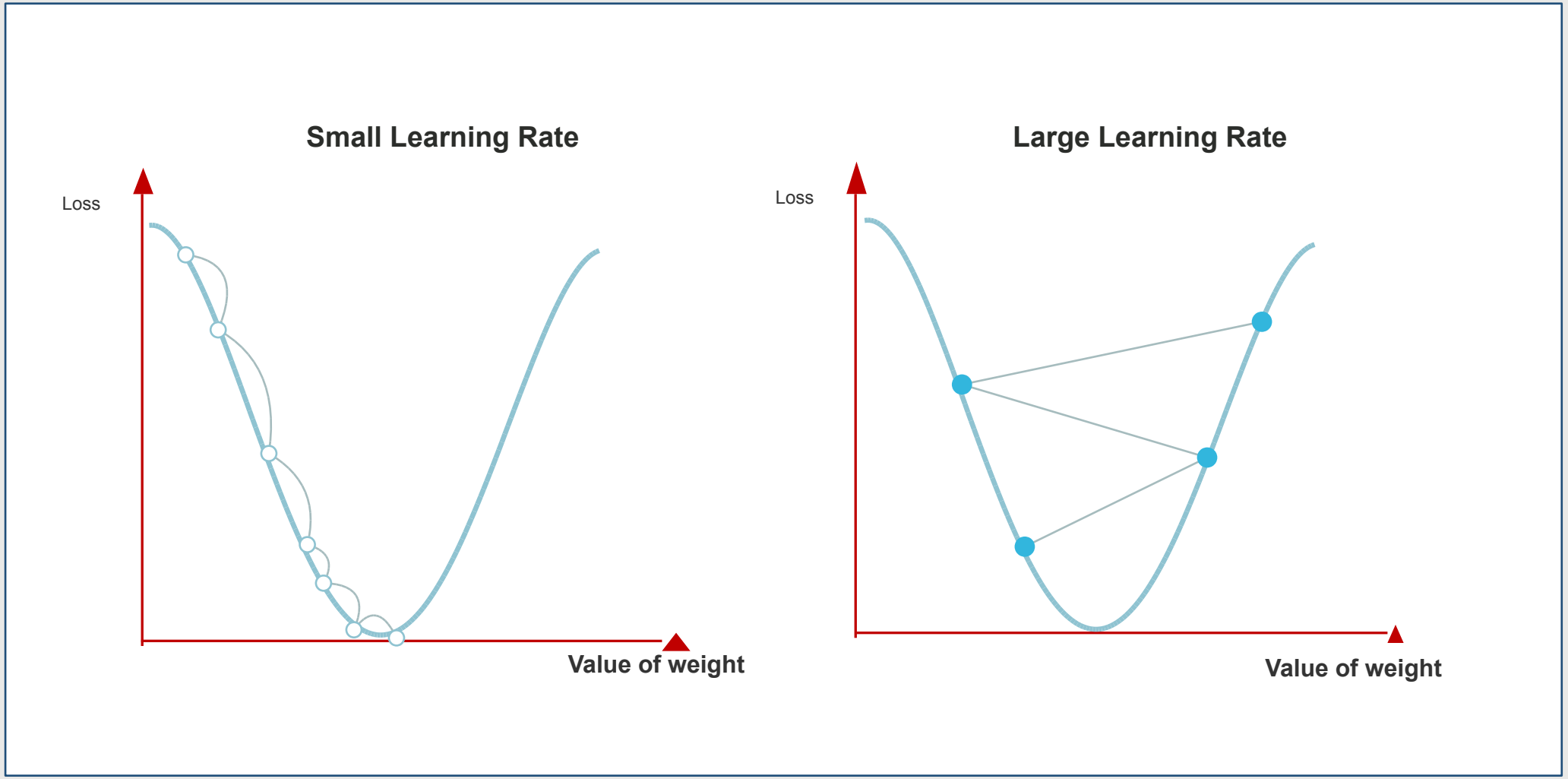
- **Least Squares** Linear regression objective function **is a convex function** (even with regularizer). This means it has **ONE Minimum** (good news).

- GD will approximatly find this minimum value but at a variable number of steps depending on the used **Learning Rate**

Follow Negative Direction of Gradient with Selected Learning Rate

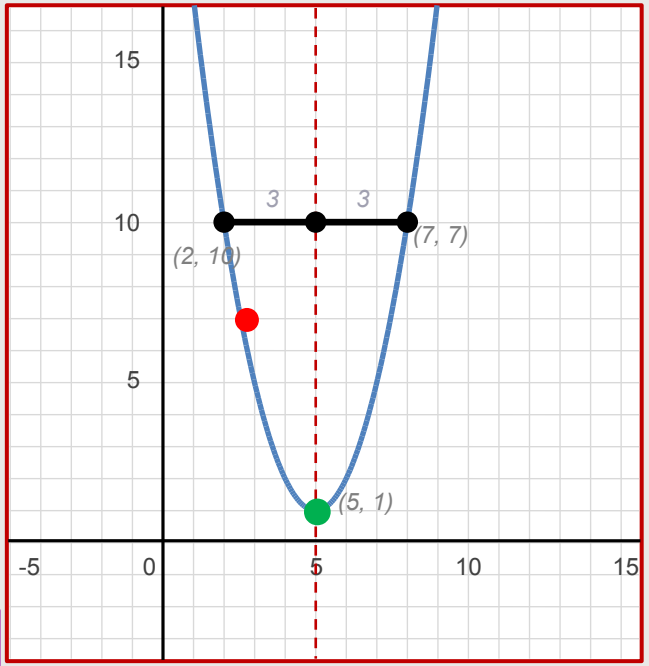


Effect of small and large Learning Rate



Example: Obtain Minimum Value of a function using Gradient Descent (GD)

Convex Function, Learning Rate =0.2, Start at X=3



step	0.2
initial X	3
delta	0.1

$f(x) = (x-5)^2 + 1$

$X = 5$

f(x) Minimum value is **1.0**

Reach Minimum after 11 Steps

x	3	3.2	3.4	3.6	3.8	4	4.2	4.4	4.6	4.8	5
f(x)	5	4.24	3.56	2.96	2.44	2	1.64	1.36	1.16	1.04	1
x-delta	2.9	3.1	3.3	3.5	3.7	3.9	4.1	4.3	4.5	4.7	4.9
f(x-delta)	5.41	4.61	3.89	3.25	2.69	2.21	1.81	1.49	1.25	1.09	1.01
x+delta	3.1	3.3	3.5	3.7	3.9	4.1	4.3	4.5	4.7	4.9	5.1
f(x+delta)	4.61	3.89	3.25	2.69	2.21	1.81	1.49	1.25	1.09	1.01	1.01
slope	-8	-7.2	-6.4	-5.6	-4.8	-4	-3.2	-2.4	-1.6	-0.8	0
next x	3.2	3.4	3.6	3.8	4	4.2	4.4	4.6	4.8	5	stop

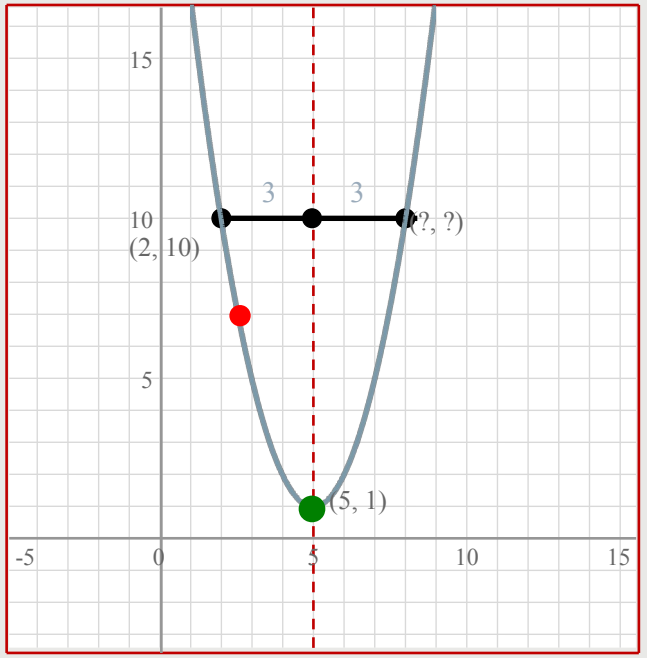
pending recreation

Convex Function, Learning Rate =0.4, Start at X=3

step	0.4
initial X	3
delta	0.1

$$f(x) = (x-5)^2 + 1$$

***X* = 5**
f(x) Minimum value is **1.0**



Reach Minimum after 6 Steps

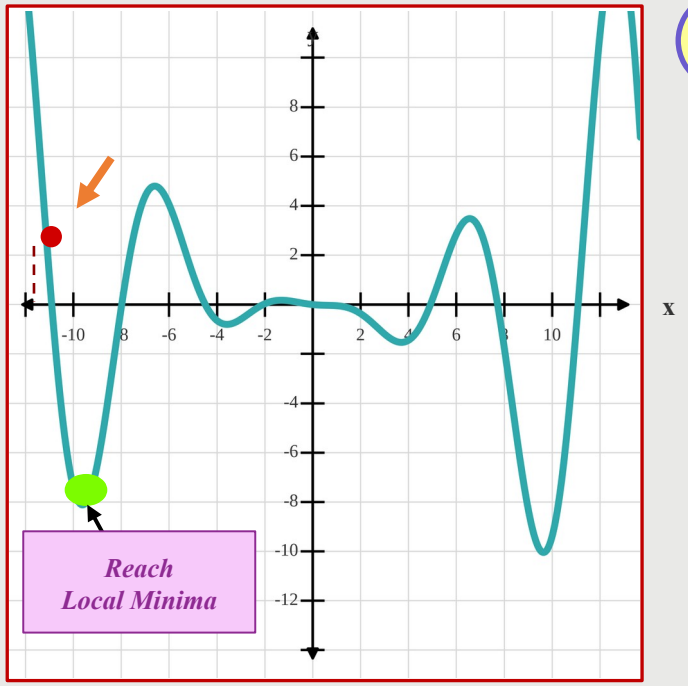
x	3	3.4	3.8	4.2	4.6	5
f(x)	5	3.56	2.44	1.64	1.16	1
x-delta	2.9	3.3	3.7	4.1	4.5	4.9
f(x-delta)	5.41	3.89	2.69	1.81	1.25	1.01
x+delta	3.1	3.5	3.9	4.3	4.7	5.1
f(x+delta)	4.61	3.25	2.21	1.49	1.09	1.01
slope	-8	-6.4	-4.8	-3.2	-1.6	0
next x	3.4	3.8	4.2	4.6	5	stop

Non-Convex Function

May reach different minimum value based on:

initial point and learning rate

$$f(x) = \frac{x^2\cos(x) - x}{10}$$



step	0.1
initial X	-11
delta	0.1

X is in between -9.7 and color red bold -9.6

f(x) is in between -8.08 and -8.011

x	-11	-10.9	-10.8	-10.7	-10.6	-10.5	-10.4	-10.3	-10.2	-10.1	-10	-9.9	-9.8	-9.7	-9.6	-9.7	-9.6	-9.7
f(x)	1.15	-0.04	-1.19	-2.26	-3.27	-4.19	-5.03	-5.77	-6.41	-6.95	-7.39	-7.72	-7.96	-8.08	-8.11	-8.08	-8.11	-8.08
x-delta	-11.1	-11	-10.9	-10.8	-10.7	-10.6	-10.5	-10.4	-10.3	-10.2	-10.1	-10	-9.9	-9.8	-9.7	-9.8	-9.7	-9.8
f(x-delta)	2.39	1.15	-0.04	-1.19	-2.26	-3.27	-4.19	-5.03	-5.77	-6.41	-6.95	-7.39	-7.72	-7.96	-8.08	-7.96	-8.08	-7.96
x+delta	-10.9	-10.8	-10.7	-10.6	-10.5	-10.4	-10.3	-10.2	-10.1	-10	-9.9	-9.8	-9.7	-9.6	-9.5	-9.6	-9.5	-9.6
f(x+delta)	-0.04	-1.19	-2.26	-3.27	-4.19	-5.03	-5.77	-6.41	-6.95	-7.39	-7.72	-7.96	-8.08	-8.11	-8.05	-8.11	-8.05	-8.11
slope	-24.3	-23.4	-22.2	-20.8	-19.3	-17.6	-15.8	-13.8	-11.8	-9.8	-7.7	-5.7	-3.6	-1.5	0.3	-1.5	0.3	-1.5
next x	-10.9	-10.8	-10.7	-10.6	-10.5	-10.4	-10.3	-10.2	-10.1	-10	-9.9	-9.8	-9.7	-9.6	-9.7	-9.6	-9.7	-9.6

pending recreation

Using Gradient Descent to train Linear Regression Model

(Problem Formulation)

Problem Formulation

Define two classes: class I and Class II

let $d(X)$ be a discriminant function = $W^T X$

if $X \in$ class I then $d(X) > 0$

if $X \in$ Class II then $d(X) < 0$

For misclassified X :

If $X \in$ class I and misclassified as Class II then $d(X)$ will be <0

If $X \in$ Class II and misclassified as Class I then $d(X)$ will be >0

Define Error function as:

$$\text{Error} = \sum_{\text{all misclassified vectors}(i)} W^T X_i \quad \Rightarrow \quad (W^T X_i) \text{ take positive and negative values}$$

X

Define δ_i as :

If $X \in$ class I and misclassified as Class II then $d(X)$ will be $<0 \Rightarrow \delta_i = -1 \Rightarrow d(X)\,\delta_i > 0$

If $X \in$ Class II and misclassified as Class I then $d(X)$ will be $>0 \Rightarrow \delta_i = +1 \Rightarrow d(X)\,\delta_i > 0$

Problem Formulation

Define δ_i as :

If $\underline{X} \in$ ***class I*** and ***misclassified*** as ***Class II*** *then* $d(\underline{X})$ *will be* $<0 \Rightarrow \delta_i = -1 \Rightarrow d(\underline{X})\delta_i > 0$

If $\underline{X} \in$ ***class II*** and ***misclassified*** as ***Class I*** *then* $d(\underline{X})$ *will be* $>0 \Rightarrow \delta_i = +1 \Rightarrow d(\underline{X})\delta_i > 0$

Redefine Error function as:

$$\text{Error} = \sum_{\text{all misclassified vectors } (i)} \delta_i \underline{W}^T \underline{X}_i \Rightarrow (\underline{W}^T \underline{X}_i) \text{ takes positive and negative values and } \delta_i \text{ has the same sign}$$

Summation is always ***positive***

Minimize the Error using ***Gradient Descent*** *with step* η

$$\underline{W}^{(updated)} = \underline{W}^{old} - \eta \frac{\partial Error}{\partial W} \quad \textbf{where} \quad \frac{\partial Error}{\partial W} = \sum_{\textit{misclassified} \text{ vectors}} \delta_i \underline{X}_i$$

$$\underline{W}^{(updated)} = \underline{W}^{old} - \eta \sum_{\textit{misclassified} \text{ vectors } \textit{ONLY} \text{ (i)}} \delta_i \underline{X}_i$$

Gradient Descent (GD)

vs

Stochastic Gradient Descent (SGD)

Gradient Descent (GD) – Stochastic Gradient Descent SGD) and Mini-Batch Gradient Descent

- Stochastic Gradient Descent (SGD) is a variant of GD which computes the gradient of the Error function with respect to a single training example
- In other words, it updates the weights using one training sample at a time
- Remember: GD updated the weight vector after passing through ALL training samples.

$$W^{(updated)} = W^{old} - \eta \delta_i X_i$$ Repeat for ALL misclassified Samples

Why using SGD

- SGD is usually more efficient than GD
- SGD converges faster than GD

Batch GD is a generalization of GD and SGD where weights are updated after passing through "Mini-Batch" of training samples with size n

If n=N (it is GD), and if n=1 (it is SGD)

Training of Linear Regression Model

usnig SGD

Solved Example

$x_1 \; x_2 \; x_3 \; x_4 \; x_5 \; x_6$

$\begin{pmatrix}1\\1\end{pmatrix} \begin{pmatrix}2\\1\end{pmatrix} \begin{pmatrix}1\\2\end{pmatrix} \begin{pmatrix}5\\4\end{pmatrix} \begin{pmatrix}6\\5\end{pmatrix} \begin{pmatrix}6\\6\end{pmatrix}$

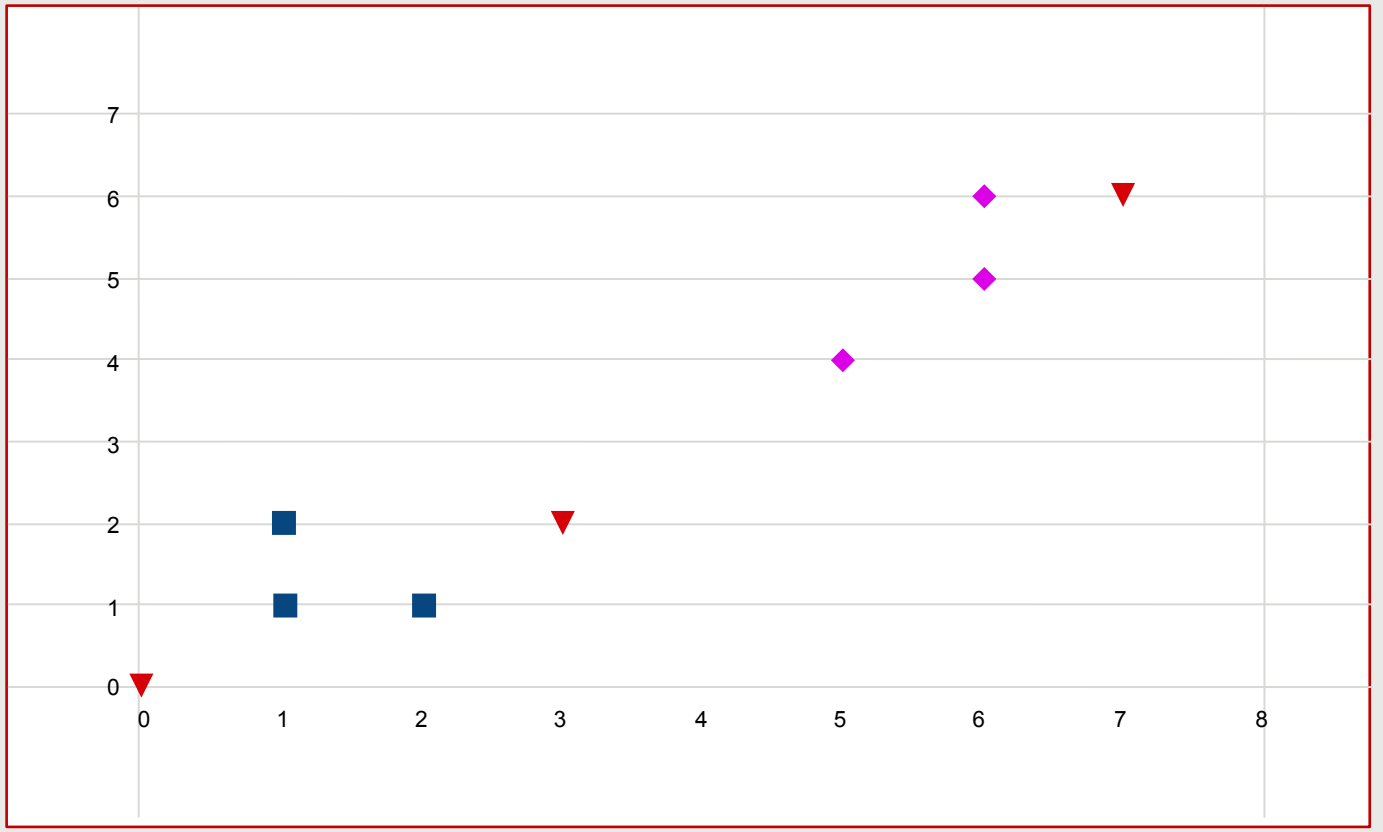
$c_1 \; c_1 \; c_1 \; c_2 \; c_2 \; c_2$

Initial W = $\begin{pmatrix}4\\-2\\-2\end{pmatrix}$ *Step η = 2*

C_1 positive, C_2 negative, Zero is unaccepted

Check classes of $z_1 \; z_1 \; z_1$

$\begin{pmatrix}0\\0\end{pmatrix} \begin{pmatrix}7\\6\end{pmatrix} \begin{pmatrix}3\\2\end{pmatrix}$



$x_1 \; x_2 \; x_3 \; x_4 \; x_5 \; x_6$

$\begin{pmatrix}1\\1\end{pmatrix}\begin{pmatrix}2\\1\end{pmatrix}\begin{pmatrix}1\\2\end{pmatrix}\begin{pmatrix}5\\4\end{pmatrix}\begin{pmatrix}6\\5\end{pmatrix}\begin{pmatrix}6\\6\end{pmatrix}$

$C_1 \; C_1 \; C_1 \; C_2 \; C_2 \; C_2$

Initial $\underline{W} = \begin{pmatrix}4\\-2\\-2\end{pmatrix}$ *Step η = 2*

C_1 positive C_2 negative Zero is unaccepted

Check classes of $Z_1 \; Z_1 \; Z_1$ $\begin{pmatrix}0\\0\end{pmatrix}\begin{pmatrix}7\\6\end{pmatrix}\begin{pmatrix}3\\2\end{pmatrix}$

Ugmented Vectors (from Top)

$x_1 \; x_2 \; x_3 \; x_4 \; x_5 \; x_6$

$\begin{pmatrix}1\\1\\1\end{pmatrix}\begin{pmatrix}1\\2\\1\end{pmatrix}\begin{pmatrix}1\\1\\2\end{pmatrix}\begin{pmatrix}1\\5\\4\end{pmatrix}\begin{pmatrix}1\\6\\5\end{pmatrix}\begin{pmatrix}1\\6\\6\end{pmatrix}$

$C_1 \; C_1 \; C_1 \; C_2 \; C_2 \; C_2$

Iteration 1

$d(x_1) = \begin{pmatrix}1&1&1\end{pmatrix}\begin{pmatrix}4\\-2\\-2\end{pmatrix} = 0 \Rightarrow$ Update Weight $\underline{W} = \begin{pmatrix}4\\-2\\-2\end{pmatrix} + 2 * \begin{pmatrix}1\\1\\1\end{pmatrix} = \begin{pmatrix}6\\0\\0\end{pmatrix} \Leftarrow$

$d(x_2) = \begin{pmatrix}1&2&1\end{pmatrix}\begin{pmatrix}6\\0\\0\end{pmatrix} = 6 \Rightarrow$ Keep same Weight $\underline{W} = \begin{pmatrix}6\\0\\0\end{pmatrix}$

$d(x_3) = \begin{pmatrix}1&1&2\end{pmatrix}\begin{pmatrix}6\\0\\0\end{pmatrix} = 6 \Rightarrow$ Keep same Weight $\underline{W} = \begin{pmatrix}6\\0\\0\end{pmatrix}$

$d(x_4) = \begin{pmatrix}1&5&4\end{pmatrix}\begin{pmatrix}6\\0\\0\end{pmatrix} = 6 \Rightarrow$ Update Weight $\underline{W} = \begin{pmatrix}6\\0\\0\end{pmatrix} - 2 * \begin{pmatrix}1\\5\\4\end{pmatrix} = \begin{pmatrix}4\\-10\\-8\end{pmatrix} \Leftarrow$

$d(x_5) = \begin{pmatrix}1&6&5\end{pmatrix}\begin{pmatrix}4\\-10\\-8\end{pmatrix} = -96 \Rightarrow$ Keep same Weight $\underline{W} = \begin{pmatrix}4\\-10\\-8\end{pmatrix}$

$d(x_6) = \begin{pmatrix}1&6&6\end{pmatrix}\begin{pmatrix}4\\-10\\-8\end{pmatrix} = -104 \Rightarrow$ Keep same Weight $\underline{W} = \begin{pmatrix}4\\-10\\-8\end{pmatrix}$

pending recreation

pending recreation

x_1	x_2	x_3	x_4	x_5	x_6
(1)	(2)	(1)	(5)	(6)	(6)
(1)	(1)	(2)	(4)	(5)	(6)
(1)	(1)	(1)	(4)	(5)	(6)
C_1	C_1	C_1	C_2	C_2	C_2

Initial $\underline{W} = \begin{vmatrix} 4 \\ -2 \\ -2 \end{vmatrix}$ ***Step η = 2***

C_1 positive, C_2 negative, **Zero** is unaccepted

Check classes of

Z_1	Z_1	Z_1
(0)	(7)	(3)
(0)	(6)	(2)

Ugmented Vectors (from Top)

(1)	(1)	(1)	(1)	(1)	(1)
(1)	(2)	(1)	(5)	(6)	(6)
(1)	(1)	(2)	(4)	(5)	(6)
(1)	(1)	(1)	(4)	(5)	(6)
C_1	C_1	C_1	C_2	C_2	C_2

Iteration 4

No change in $\underline{W}$

Stop Iterations

$d(x_1) = 1\ 1\ 1 \begin{vmatrix} 14 \\ -4 \\ 0 \end{vmatrix} = 10$ ⇒ Keep same Weight $\underline{W} = \begin{vmatrix} 14 \\ -4 \\ 0 \end{vmatrix}$

$d(x_2) = 1\ 2\ 1 \begin{vmatrix} 14 \\ -4 \\ 0 \end{vmatrix} = 6$ ⇒ Keep same Weight $\underline{W} = \begin{vmatrix} 14 \\ -4 \\ 0 \end{vmatrix}$

$d(x_3) = 1\ 1\ 2 \begin{vmatrix} 14 \\ -4 \\ 0 \end{vmatrix} = 10$ ⇒ Keep same Weight $\underline{W} = \begin{vmatrix} 14 \\ -4 \\ 0 \end{vmatrix}$

$d(x_4) = 1\ 5\ 4 \begin{vmatrix} 14 \\ -4 \\ 0 \end{vmatrix} = -6$ ⇒ Keep same Weight $\underline{W} = \begin{vmatrix} 14 \\ -4 \\ 0 \end{vmatrix}$

$d(x_5) = 1\ 6\ 5 \begin{vmatrix} 14 \\ -4 \\ 0 \end{vmatrix} = -10$ ⇒ Keep same Weight $\underline{W} = \begin{vmatrix} 14 \\ -4 \\ 0 \end{vmatrix}$

$d(x_6) = 1\ 6\ 6 \begin{vmatrix} 14 \\ -4 \\ 0 \end{vmatrix} = -112$ ⇒ Keep same Weight $\underline{W} = \begin{vmatrix} 14 \\ -4 \\ 0 \end{vmatrix}$

x_1 x_2 x_3 x_4 x_5 x_6

$\begin{pmatrix}1\\1\end{pmatrix}$ $\begin{pmatrix}2\\1\end{pmatrix}$ $\begin{pmatrix}1\\2\end{pmatrix}$ $\begin{pmatrix}5\\4\end{pmatrix}$ $\begin{pmatrix}6\\5\end{pmatrix}$ $\begin{pmatrix}6\\6\end{pmatrix}$

C_1 C_1 C_1 C_2 C_2 C_2

Initial $\underline{W} = \begin{pmatrix}4\\-2\\-2\end{pmatrix}$ ***Step η = 2***

C_1 *positive* C_2 *negative* ***Zero*** *is unaccepted*

Check classes of Z_1 Z_1 Z_1

$\begin{pmatrix}0\\0\end{pmatrix}$ $\begin{pmatrix}7\\6\end{pmatrix}$ $\begin{pmatrix}3\\2\end{pmatrix}$

$\underline{W} = \begin{pmatrix}14\\-4\\0\end{pmatrix}$

Ugmented Vectors (from Top)

x_1 x_2 x_3 x_4 x_5 x_6

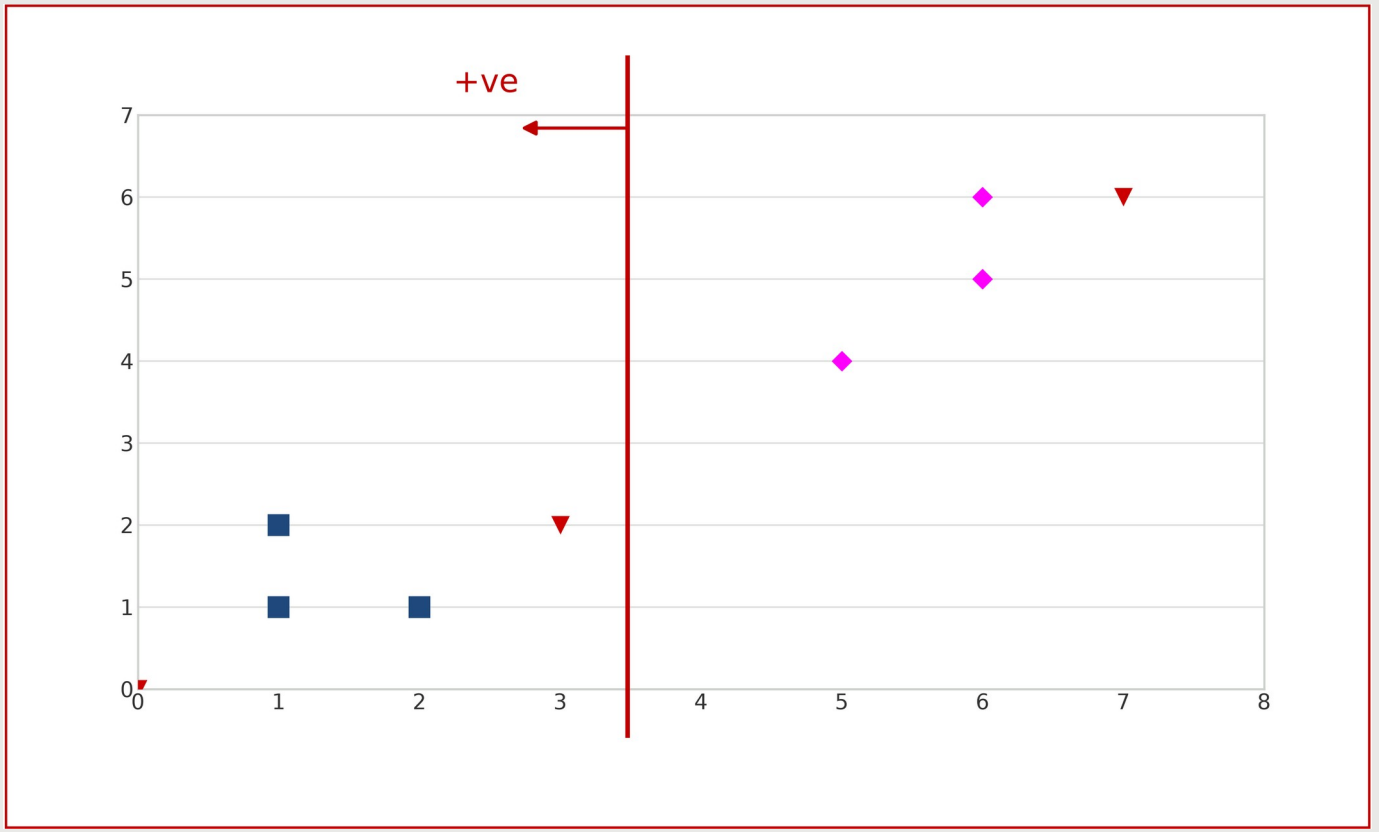
$\begin{pmatrix}1\\1\\1\end{pmatrix}$ $\begin{pmatrix}1\\2\\1\end{pmatrix}$ $\begin{pmatrix}1\\1\\2\end{pmatrix}$ $\begin{pmatrix}1\\5\\4\end{pmatrix}$ $\begin{pmatrix}1\\6\\5\end{pmatrix}$ $\begin{pmatrix}1\\6\\6\end{pmatrix}$

C_1 C_1 C_1 C_2 C_2 C_2

Iteration 4

No change in $\underline{W}$

Stop *Iterations*



x_1 x_2 x_3 x_4 x_5 x_6

$\begin{pmatrix}1\\1\end{pmatrix}$ $\begin{pmatrix}2\\1\end{pmatrix}$ $\begin{pmatrix}1\\2\end{pmatrix}$ $\begin{pmatrix}5\\4\end{pmatrix}$ $\begin{pmatrix}6\\5\end{pmatrix}$ $\begin{pmatrix}6\\6\end{pmatrix}$

C_1 C_1 C_1 C_2 C_2 C_2

Initial $\underline{\mathbf{W}} = \begin{pmatrix}4\\-2\\-2\end{pmatrix}$ ***Step η = 2***

C_1 positive C_2negative Zero is unaccepted

Check classes of Z_1 Z_1 Z_1

$\begin{pmatrix}0\\0\end{pmatrix}$ $\begin{pmatrix}7\\6\end{pmatrix}$ $\begin{pmatrix}3\\2\end{pmatrix}$

Ugmented Vectors (from Top)

x_1 x_2 x_3 x_4 x_5 x_6

$\begin{pmatrix}1\\1\\1\end{pmatrix}$ $\begin{pmatrix}1\\2\\1\end{pmatrix}$ $\begin{pmatrix}1\\1\\2\end{pmatrix}$ $\begin{pmatrix}1\\5\\4\end{pmatrix}$ $\begin{pmatrix}1\\6\\5\end{pmatrix}$ $\begin{pmatrix}1\\6\\6\end{pmatrix}$

C_1 C_1 C_1 C_2 C_2 C_2

Check Classes

$d(Z_1) = (1\ 0\ 0)\begin{pmatrix}14\\-4\\0\end{pmatrix} = 14$ ⇒ **Positive > 0** **Class I**

$d(Z_2) = (1\ 7\ 6)\begin{pmatrix}14\\-4\\0\end{pmatrix} = -14$ ⇒ **Negative < 0** **Class II**

$d(Z_2) = (1\ 3\ 2)\begin{pmatrix}14\\-4\\0\end{pmatrix} = -2$ ⇒ **Negative < 0** **Class II**

pending recreation

pending recreation

pending recreation

pending recreation